

PC BUILD STORE - OBJECT ORIENTED PROGRAMMING PROJECT

FINAL REPORT

GROUP MEMBERS

1. Abdulhanan (rehanazizabdulhanan@gmail.com)
2. Ahmad Mushtaq (2501803@students.au.edu.pk)
3. Aitzaz (2501805@students.au.edu.pk)
4. Haroon (haroonikram2512@gmail.com)

1. INTRODUCTION

PC Build Store is a desktop application designed to help users configure, compare, and purchase custom PC builds. The application provides a comprehensive catalog of 78+ components across 6 categories (CPU, GPU, RAM, Storage, PSU, Motherboard) with real pricing data from the Pakistani market.

The system allows users to:

- Browse and search PC components by category
- Build custom PCs with real-time compatibility filtering
- Upgrade existing builds with smart compatibility checks
- Purchase builds and receive printable/saveable receipts
- Get AI-powered build suggestions through an integrated chat bot

The application is built using Java 17 with Swing GUI and MariaDB for data persistence, following object-oriented design principles throughout.

2. PROBLEM STATEMENT

Building a custom PC is a complex task that requires knowledge of hardware compatibility, pricing, and performance metrics. Users often struggle with:

- Identifying compatible components across different categories
- Comparing prices from multiple vendors
- Understanding technical specifications (socket types, DDR generations, power requirements)
- Getting expert advice on optimal configurations

Our solution addresses these challenges by providing an intelligent PC configurator with automated compatibility checks, real-time pricing, and AI-powered recommendations.

3. OBJECTIVES

Primary Objectives:

- Build a user-friendly PC configurator with drag-and-drop simplicity
- Implement automated compatibility checking across all component types
- Provide real-time pricing from Pakistani vendors (zahcomputers.pk, galaxy.pk)
- Integrate AI chat assistance for build recommendations
- Generate professional receipts for purchase records

Secondary Objectives:

- Implement a dark theme UI for modern aesthetics
- Cache product images for fast loading
- Support streaming AI responses for interactive chat
- Generate revenue and performance analytics reports
- Provide modular architecture for easy maintenance

4. SYSTEM ARCHITECTURE

The application follows a layered MVC (Model-View-Controller) architecture:

PRESENTATION LAYER (GUI)

-
- DashboardGUI: Main navigation hub with hero section, stats, and chat
 - BuildCatalogGUI: 6-slot PC builder with category tabs and product grid
 - BuildUpgradesGUI: Upgrade system with compatibility checks
 - BillingGUI: Purchase flow with receipt generation
 - ReportGUI: Revenue and performance analytics charts
 - AddPartDialog: Modal form for adding components
 - ChatSettingsDialog: AI configuration dialog

BUSINESS LOGIC LAYER (DAO)

-
- BillDAO: Purchase operations and receipt generation
 - BuildDAO: Build CRUD operations and part management
 - CategoryDAO: Category retrieval and caching
 - PartDAO: Component queries, search, and filtering
 - GPUOptionDAO: GPU-specific upgrade options
 - ReportDAO: Analytics aggregation queries

DATA ACCESS LAYER (JDBC)

-
- DBConnection: MariaDB connection pool management
 - JDBC drivers for MySQL/MariaDB communication

MODEL LAYER (POJOs)

-
- Bill: Purchase record with build reference and pricing
 - Build: PC configuration with total price and score
 - BuildPart: Junction table for build-component relationships
 - Category: Component category (CPU, GPU, etc.)
 - Part: Hardware component with full specifications
 - GPUOption: GPU-specific upgrade option

UTILITY LAYER

-
- ImageCache: 3-tier caching (memory, disk, HTTP) with LRU eviction
 - ChatService: SSE streaming for OpenAI-compatible APIs
 - ChatConfig: Persistent settings management
 - Components: Theme-consistent UI factories

DATA FLOW:

User Interface -> DAO Layer -> JDBC -> MariaDB -> DAO -> UI Update

5. MODULE DISTRIBUTION

Module	Lead	Members	GUI Work	Backend Work	
Database Work					
-----	-----	-----	-----	-----	---

Dashboard	Abdulhanan	All	Hero, Stats	Chat Streaming	
Stats Queries					
Build Configurator	Haroon	All	Cart, Grid	Part Filtering	
Part Queries					
Build Upgrades	Aitzaz	All	Upgrade UI	Compatibility	
Build Updates					
Billing	Ahmad Mushtaq	All	Receipt View	Print/Save	
Bill Inserts					
Reports	Abdulhanan	All	Charts	Aggregation	
Report Queries					
AI Chat	Abdulhanan	All	Settings Dlg	SSE Streaming	
N/A					

6. INTEGRATION STRATEGY

The integration was performed in phases:

Phase 1 - Database Setup:

- Created MariaDB schema with 6 tables
- Populated 78+ components with real pricing data
- Established JDBC connection pool

Phase 2 - Module Development:

- Each team member developed their assigned module independently
- Used consistent theme (Theme.java) for uniform appearance
- Shared DAO layer for database operations

Phase 3 - Cross-Module Integration:

- Dashboard links to all modules via navigation buttons
- Build Catalog passes selected builds to Upgrades module
- Billing receives completed builds for purchase processing
- Reports aggregate data from all modules

Phase 4 - Feature Completion:

- Added AI chat integration with streaming responses
- Implemented image caching for product photos
- Added receipt print and save functionality
- Created add part dialog for manual component entry

7. IMPLEMENTATION DETAILS

7.1 Dashboard Module (Abdulhanan)

The dashboard serves as the main navigation hub with:

- Hero banner with gradient background
- Statistics cards (total builds, parts, revenue, purchases)
- Featured parts grid with lazy-loaded images
- AI chat panel with streaming SSE responses
- Navigation buttons to all modules

Key Implementation:

- GridBagLayout for responsive stat cards
- FontMetrics-based text height calculation for chat bubbles
- Row+bubble layout for message display
- KoboldCPP integration via OpenAI-compatible API

7.2 Build Configurator Module (Haroon)

The build configurator provides:

- 6-slot PC builder (CPU, GPU, RAM, Storage, PSU, Motherboard)
- Category tabs for filtering components
- Product grid with images and specs
- Cart sidebar showing selected parts
- Real-time total price and performance score

Key Implementation:

- Motherboard compatibility filtering by socket type
- DDR generation matching between CPU and RAM
- Cart state management across category switches
- Image loading via ImageCache with parallel fetching

7.3 Build Upgrades Module (Aitzaz)

The upgrade system allows:

- Loading existing builds for modification
- 6-slot upgrade interface with current parts displayed
- Compatibility checking (socket, DDR, wattage)
- Apply upgrades button to save changes

Key Implementation:

- Socket compatibility matrix for CPU-Motherboard matching
- DDR generation validation for CPU-RAM compatibility
- Wattage calculation for PSU headroom checking
- Build part replacement with score recalculation

7.4 Billing Module (Ahmad Mushtaq)

The billing system provides:

- Build summary display with all selected parts
- Purchase processing with database storage
- Receipt view with formatted HTML output
- Print receipt via java.awt.print.PrinterJob
- Save receipt as HTML file via JFileChooser

Key Implementation:

- Printable interface for receipt formatting
- HTML receipt generation with table layout
- JFileChooser with null parent to avoid modal blocking
- Purchase date formatting for receipt display

7.5 Reports Module (Abdulhanan)

The reports module generates:

- Revenue over time chart (line chart)
- Parts distribution by category (pie chart)
- Performance metrics (bar chart)
- Summary statistics cards

Key Implementation:

- Custom JPanel painting for chart rendering
- Date-based revenue aggregation
- Category-wise part counting
- Performance score averaging

7.6 AI Chat Module (Abdulhanan)

The chat system provides:

- OpenAI-compatible API integration
- Streaming SSE response parsing
- KoboldCPP backend support (Qwen3 model)
- Configurable settings (base URL, API key, model)

Key Implementation:

- java.net.http.HttpClient for streaming
- Server-Sent Events parsing with chunked reading
- Dual-field JSON parsing (content + reasoning_content)
- FontMetrics-based bubble height calculation
- Settings persistence in ~/.pcbuidstore/config.json

=====

8. DATABASE DESIGN

=====

8.1 Table: categories

Column	Type	Constraint
category_id	INT	PK AUTO_INCREMENT
name	VARCHAR(50)	NOT NULL

8.2 Table: parts

Column	Type	Constraint
part_id	INT	PK AUTO_INCREMENT
category_id	INT	FK -> categories
brand	VARCHAR(50)	NOT NULL
name	VARCHAR(100)	NOT NULL
price	INT	NOT NULL
performance_score	INT	DEFAULT 0
socket_type	VARCHAR(20)	NULL
ddr_generation	INT	NULL

core_count	INT	NULL
clock_speed	DOUBLE	NULL
vram	INT	NULL
memory_speed	INT	NULL
capacity	INT	NULL
read_speed	INT	NULL
wattage	INT	NULL
efficiency	VARCHAR(10)	NULL
image_path	VARCHAR(500)	NULL

8.3 Table: builds

Column	Type	Constraint
build_id	INT	PK AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
total_price	INT	DEFAULT 0
total_score	INT	DEFAULT 0

8.4 Table: build_parts

Column	Type	Constraint
build_id	INT	FK -> builds, PK
category_id	INT	FK -> categories, PK
part_id	INT	FK -> parts
price_at_add	INT	NOT NULL

8.5 Table: bills

Column	Type	Constraint
bill_id	INT	PK AUTO_INCREMENT
build_id	INT	FK -> builds
final_price	INT	NOT NULL
final_score	INT	NOT NULL
purchase_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

9. TESTING

9.1 Dashboard Testing

- Verified stats cards display correct counts
- Tested navigation buttons to all modules
- Confirmed chat streaming with KoboldCPP server
- Validated featured parts image loading

9.2 Build Configurator Testing

- Tested category tab switching and part filtering
- Verified 6-slot selection (one per category)
- Confirmed real-time price/score calculation
- Tested cart sidebar updates
- Validated motherboard compatibility filtering

9.3 Build Upgrades Testing

- Tested loading existing builds
- Verified compatibility checks (socket, DDR, wattage)
- Confirmed part replacement functionality
- Tested apply upgrades button
- Validated score recalculation

9.4 Billing Testing

- Tested purchase processing and database insertion
- Verified receipt view with correct part listing
- Tested print dialog and printer output
- Confirmed save receipt as HTML file
- Validated JFileChooser modal behavior

9.5 Reports Testing

-
- Verified revenue chart data from database
 - Tested parts distribution chart
 - Confirmed performance metrics display
 - Validated date range filtering

9.6 Chat Testing

-
- Tested streaming response parsing
 - Verified KoboldCPP integration
 - Confirmed settings persistence
 - Tested error handling for server unavailability

10. RESULTS

The PC Build Store application successfully delivers:

- A comprehensive catalog of 78+ PC components with real pricing
- Automated compatibility checking across all component types
- Real-time build cost and performance calculation
- Professional receipt generation with print/save options
- AI-powered build assistance via streaming chat
- Dark theme UI with modern aesthetics
- Responsive layout with custom painting

The application handles edge cases gracefully:

- Database connection failures with fallback messages
- Image loading failures with placeholder icons
- Server unavailability for chat with user notification
- Invalid user inputs with validation messages

11. CONCLUSION

The PC Build Store project demonstrates effective application of object-oriented programming principles including:

- Encapsulation: Model classes with private fields and public getters
- Inheritance: JPanel anonymous inner classes for custom components
- Polymorphism: DAO pattern for interchangeable data access
- Abstraction: Separation of concerns across layers
- Composition: GUI panels composed of specialized sub-components

The modular architecture allowed parallel development by team members while maintaining code consistency through shared theme and DAO layers.

Key technical achievements:

- Streaming AI integration with KoboldCPP
- 3-tier image caching system
- Real-time compatibility checking
- Professional receipt generation

The project provides a solid foundation for future enhancements such as price comparison across multiple vendors, user accounts with build history, and cloud-based AI recommendations.

END OF REPORT